AI DRIVEN PENETRATION TESTING ASSISTANT

Saif Salah Ud Din Haider
Nabeel Hassan
Subhan Ahmed

Supervisor: Mam Sumaira Sarwar

FALL 2025

Department of Computer Science

Capital University of Science & Technology, Islamabad

Table of Contents

List of Tables

List of Figures

Chapter 1

Introduction

This chapter provides a brief summary of project scope, project specification, a comparison study with the available existing solutions and existing tools and technologies may be used for the development of this project. This chapter also includes a project work breakdown structure and a proposed timeline.

1.1 Project Introduction

In the rapidly evolving landscape of cybersecurity, penetration testing (pentesting) plays a critical role in identifying and mitigating vulnerabilities before malicious attackers exploit them. However, traditional pentesting requires deep technical knowledge of numerous tools, complex command-line interfaces, and attack methodologies, which makes the process time-consuming and less accessible to beginners.

To address these challenges, our Final Year Project (FYP) proposes the development of an **AI-powered Penetration Testing Assistant** that integrates Large Language Models (LLMs) such as ChatGPT with widely used open-source security tools. This assistant enables users to interact with pentesting tools using **natural language commands**. For example, instead of manually typing nmap -sV 192.168.100.11, a user can simply request: *"Scan 192.168.100.11 for open ports and services"*. The AI will then translate this request into the appropriate command, execute the tool in the backend, analyze the output, and provide human-readable results

1.2 Existing Examples / Solutions

In recent years, several platforms and tools have attempted to combine automation, artificial intelligence, and penetration testing. These solutions aim to simplify vulnerability discovery, improve reporting, and reduce the reliance on manual expertise. However, most of them either focus on limited functionality or require expensive licensing. Some notable examples include:

1. PentestGPT

PentestGPT is a research-driven project that integrates GPT models with penetration testing workflows. It assists security professionals in reconnaissance, vulnerability scanning, and exploitation suggestions. However, PentestGPT is still experimental and primarily relies on external tools for execution. Its focus is more on assisting human testers with reasoning rather than providing full automation.

2. Horizon3.ai (NodeZero)

NodeZero is an autonomous penetration testing platform that continuously scans networks and validates vulnerabilities through safe exploitation. It functions more as an enterprise solution and requires a subscription. While powerful, it is not open-source and thus not easily adaptable for research or academic projects.

3. Pentest-Tools.com

Pentest-Tools.com is a web-based commercial solution that provides automated reconnaissance, scanning, and exploitation through a cloud dashboard. It allows security teams to launch common tests such as subdomain enumeration, port scanning, and SQL injection detection. However, it is paid software and does not provide integration with AI models for natural language interaction.

Summary of Gap

AI reasoning assistants (e.g., PentestGPT) → provide guidance, but don't execute. Automated scanning platforms (e.g., Pentest-Tools, NodeZero) → execute tests, but lack AI reasoning and flexibility.

Our proposed project fills the gap by combining both:

AI (LLM) → to translate user natural language into structured tool commands. Backend Integration → to actually execute tools like Nmap, SQLmap Decision Loop → AI analyzes outputs and suggests the next step.

Business Scope

The proposed project is mainly designed for educational use, beginners in cybersecurity, and small lab environments. Its purpose is not to replace professional pentesting frameworks but to provide an intelligent, easy-to-use assistant that helps users understand and practice penetration testing concepts.

Target Audience

Students and Beginners in Cybersecurity

1. The system allows learners to use real pentesting tools without needing to memorize complex commands.
2. By interacting in natural language (e.g., "Scan this IP for open ports"), students can see how tools like Nmap, SQLmap, or Nikto are used in real practice.

Educational Labs & Universities

3. Universities can use this project in ethical hacking and network security courses.
4. It provides a safe, controlled platform where students learn both AI and security fundamentals.

Entry-Level Security Enthusiasts

5. People who are curious about penetration testing but have little hands- on experience can use the assistant as a training companion.

6. Instead of struggling with syntax errors or tool configurations, they can focus on understanding results and attack flow.

7. Small Academic Projects / Research Groups

8. This framework can serve as a foundation for future research in AI- assisted cybersecurity.

9. It encourages experiments on tool integration, AI decision-making, and workflow automation.

1.3 Useful Tools and Technologies

1. Passive Information Gathering Tools

- These tools are used to collect information about a target without directly interacting with it, which reduces the chance of detection.
- The Harvester – Used for gathering emails, subdomains, hosts, employee names, and open ports from public sources like search engines and PGP key servers.
- Dig – A DNS lookup utility used to query domain name servers and gather DNS records (A, MX, TXT, NS).
- Amass — an open-source DNS/subdomain enumeration and attack- surface discovery tool (passive + active, bruteforce, cert-transparency and OSINT integrations).

2. Active Scanning Tools

- These tools directly interact with the target to discover live hosts, open ports, and services.

- Nmap: The most widely used network scanning tool, capable of detecting open ports, services, versions, and operating systems.

- Masscan: an ultra-fast asynchronous TCP SYN port scanner for Internet-scale scans (sends raw packets at very high rates; nmap-like syntax, use with care and permission).

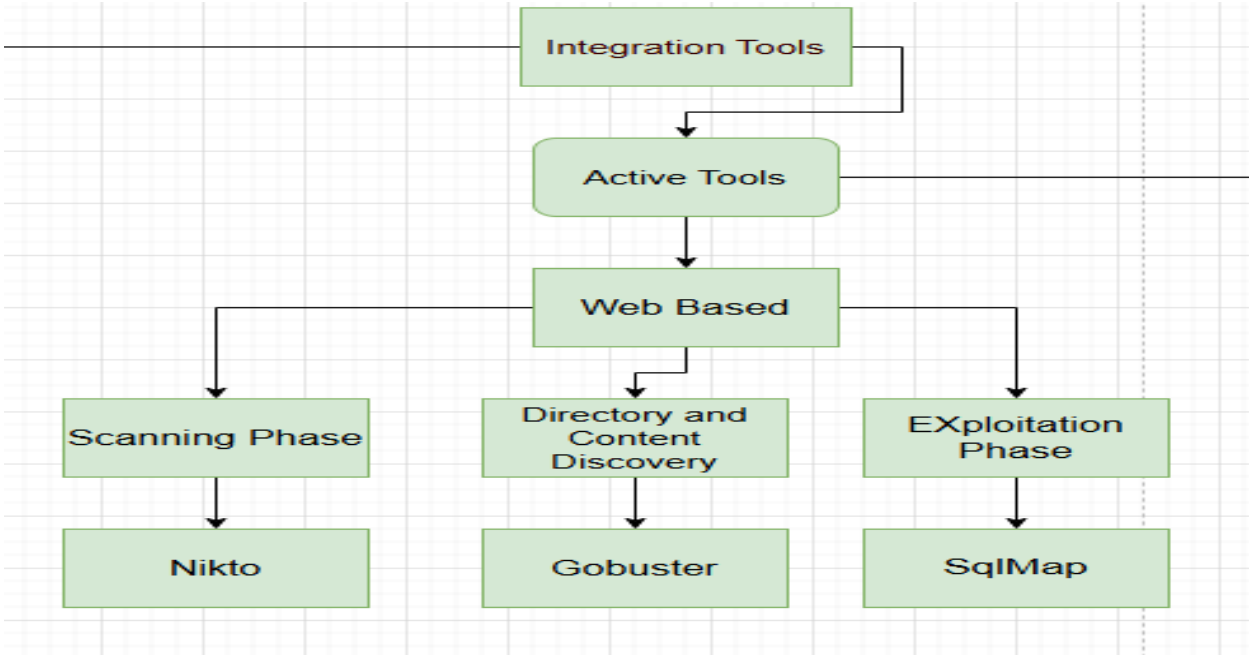
3. Vulnerability Assessment Tools

- OpenVAS**:** A full-featured vulnerability scanner that checks for known vulnerabilities in network services and systems.

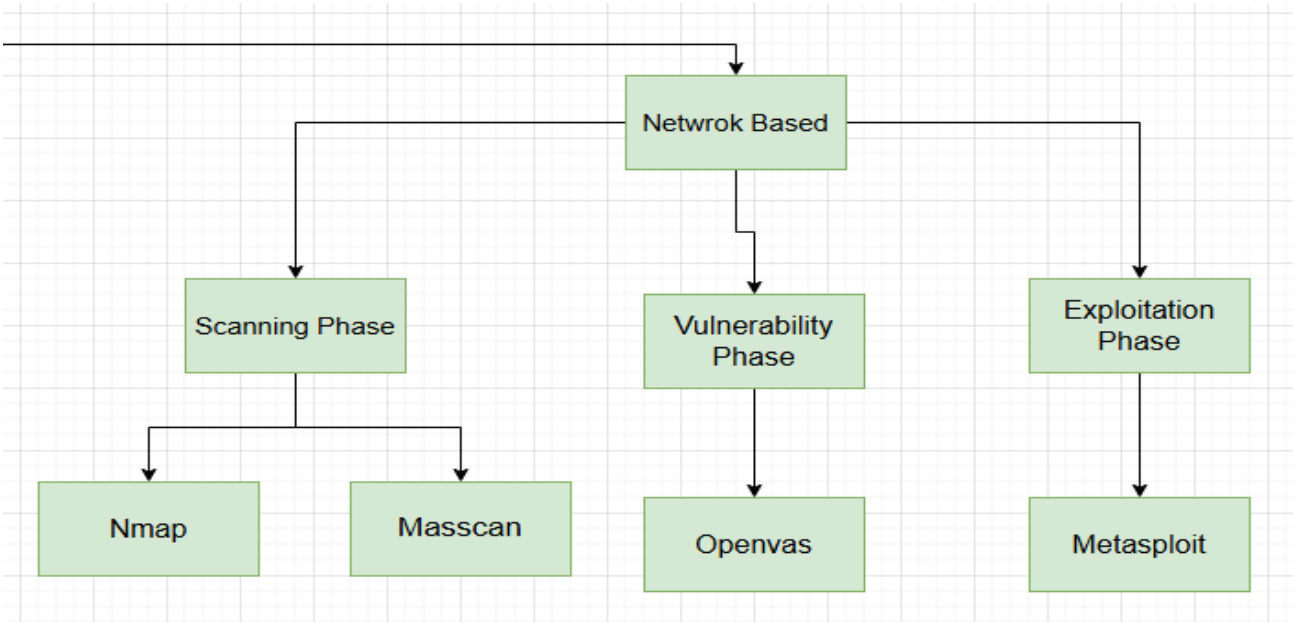
4. Web Scanning

- Nikto**:** A web server scanner that detects outdated software, dangerous files, and potential misconfigurations.
- SQLmap**:** An automated tool for detecting and exploiting SQL injection vulnerabilities in web applications, and for taking over database servers.
- WPScan**:** A WordPress vulnerability scanner used to detect insecure plugins, themes, and weak configurations.

Web Based



Network Based



1.4 Testing Type

1 Unit Testing

Unit testing ensures that each individual component of the system functions correctly and reliably.

In the Proposed System:

Each core module — such as the AI command generator, tool executor (integrated with Nmap, Nikto, and Dig), and the output analyzer — was tested independently.

For example, the AI-based command generator was evaluated to confirm that it produced valid and accurate command syntax for tasks like port scanning and web vulnerability detection.

2 Integration Testing

Integration testing verifies that all modules of the system work together seamlessly as a single unit.

In the Proposed System:

After successful unit testing, the modules were combined. The AI-generated commands were passed to the backend executor, which then used the appropriate tool (e.g., Nmap, Nikto, or Dig). The results were analyzed by the output analyzer.

Integration testing confirmed smooth communication and consistent data flow between these components.

3 System Testing

System testing validates the overall functionality and performance of the complete system against the defined requirements.

In the Proposed System:

End-to-end testing was performed across the entire workflow:

User Query → AI Command Generation → Tool Execution → Result Display.

This ensured that the system accurately performed network scanning, web vulnerability detection, and DNS analysis without errors or performance issues.

4 User Acceptance Testing (UAT)

User Acceptance Testing ensures that the final system meets user expectations in terms of usability, accuracy, and responsiveness.

In the Proposed System:

The system was tested by students and supervisors to evaluate its usability. Feedback on interface clarity, response accuracy, and ease of command generation was collected and implemented to improve the UI and overall user experience.

5 Security Testing

Security testing ensures the system operates safely and ethically, especially since it involves pentesting tools.

In the Proposed System:

Security testing verified that the system:

- Executes only authorized and ethical scans.
- Prevents harmful or destructive commands.
- Ensures protection of user data and scan results.
- Identifies vulnerabilities without causing damage to the target systems.

6 Performance Testing

Performance testing measures the system's efficiency, stability, and response time under different conditions.

In the Proposed System:

Tests were conducted to evaluate:

- Command generation speed by the AI module.
- Execution time of tools such as Nmap, Nikto, and Dig.
- System response time when handling multiple simultaneous requests.

Model Fine Tuning

Table 1 Model Fine Tuning

LLM State	Analogy	The Problem	The Solution (Fine-Tuning)
Pre-trained Llama 3 8B	**A Smart Student:** Knows general English, but has never taken a cybersecurity course.	If you ask, "Scan the IP 10.0.0.5," it might say: "I apologize, I cannot generate commands to harm others." (Too safe) OR It might "hallucinate" the wrong command: nmap scan 10.0.0.5 --ports all (Wrong syntax/unsafe).	**You need it to be a Specialist.** Fine-tuning gives it the specific instruction: "When a user asks for a scan, reply with only the correct nmap syntax, like a professional." It improves **Accuracy**, reduces **Hallucination**, and breaks the generic safety guardrails (for ethical, in-scope commands).

- This is the example of one tool but we use many tools so its increase our final Year Project Scope. If we fine tune the model for every tool
- Fine-tuning is important because it changes the model from a general "assistant" to a specialized "command generator," ensuring the commands are syntactically correct and relevant to the penetration testing context.

1.5 Dataset

Dataset strategy (Hybrid approach)

- Foundation (public sources): Harvest canonical command examples and documentation from official tool documentation, reputable tutorials, GitHub repositories, and curated datasets (Kaggle
- Hugging Face to form the skeleton of the dataset.
- Core (manual curation): Manually produce high-quality, canonical NL→CLI pairs targeted to the project use cases. Each canonical command will be paired with 5–10 natural-language paraphrases to improve model robustness.
- Augmentation: Apply template-based generation and safe paraphrasing to instantiate realistic inputs (IPs, domains, URLs, ports) while preserving canonical outputs for training.
- Safety filtering: Annotate each entry with metadata (tool, risk_level, safe_to_execute) and exclude or de-scope destructive or exploit payloads unless explicit authorization is provided

1.6 Format & size

- Data will be stored as JSONL records with the schema:
- { instruction, input, output, tool, risk_level, safe_to_execute }.
- Initial target: a focused seed dataset of 100–200 high-quality NL→CLI pairs for Nmap and sqlmap, to be expanded iteratively.

1.7 Training & validation

1. Fine-tune an instruction/casual LLM (using LoRA/adapters if GPU resources are limited) with a consistent prompt structure:
2. Instruction: ...
3. Input: ...
4. Command:
5. Evaluate using both textual and functional metrics:
6. Exact Match (normalized)
7. Syntax checks (balanced quotes, no unresolved placeholders)
8. Sandboxed execution tests against lab VMs (authorized targets only) to verify functional correctness
9. Human review for ambiguous or high-risk outputs

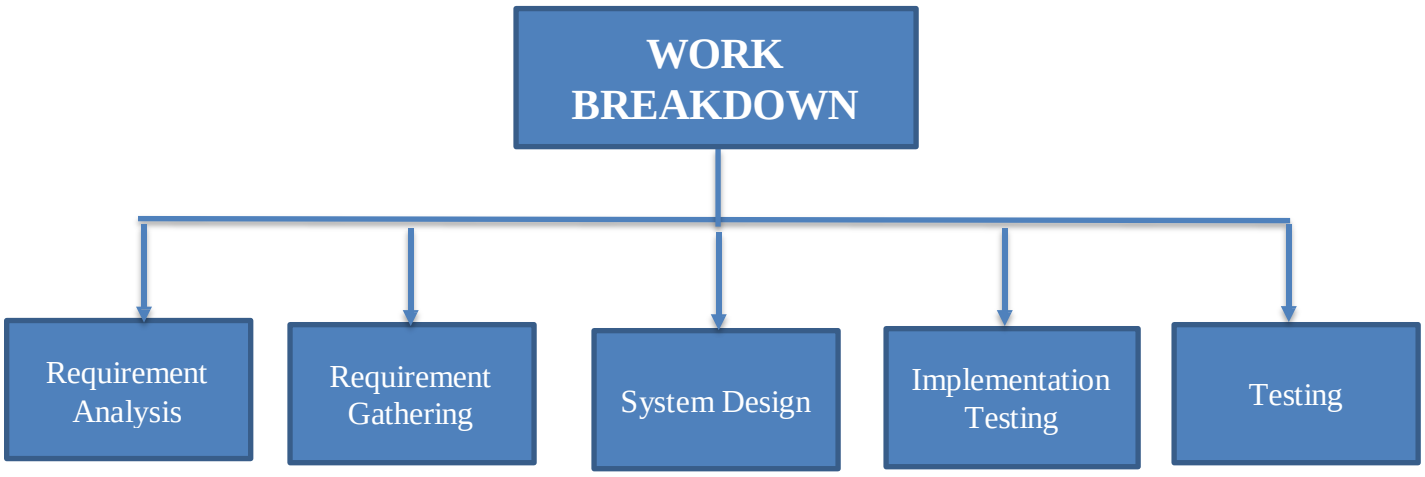
1.8 Safety & deployment controls

1. All automated executions are restricted to an isolated lab network and whitelisted IP ranges.
2. A pre-execution validator will block dangerous flags or unauthorized targets and will require human approval when necessary.
3. Maintain full logging and an audit trail for generated commands and execution outcomes.

1.9 Deliverables

1. dataset_v1.jsonl (seed dataset) and an accompanying spreadsheet index for review.
2. Training / fine-tuning configuration and model checkpoint.
3. Evaluation report (EM, functional success rate, syntax error rate) including representative failure cases.
4. Demo illustrating NL input → generated command → sandboxed execution results (for safe, authorized tests)

1.10 Project Work Break Down



1.11 Project Time Line

	Sept	Oct	Nov	Dec	Jan	Feb
Requirement Gathering	✓	✓	✓			
Requirement Analysis				✓	✓	✓
Design						
Implementation						
Testing						

Chapter 2

2.1 Requirement Specification and Analysis

The main goal of this phase is to identify the functional and non-functional requirements of the AI-Driven Penetration Testing Assistant. This chapter describes how the system will operate, how users and components interact, and what constraints ensure security, performance, and scalability. These requirements serve as a foundation for system design, modeling, and implementation.

2.2 Functional Requirements

Functional requirements define the specific features, operations, and interactions that the system must perform. They describe how each module of the AI-Driven Penetration Testing Assistant contributes to safe and automated vulnerability scanning.

Table 2.2: Functional Requirements

S.No.	Functional Requirement	Type	Status
1	User interaction through a web interface for command input.	User Interaction	To be Implemented
2	Natural Language Processing (NLP) to interpret user intent (tool, target, purpose).	AI Command Understanding	To be Implemented
3	Generate structured JSON output containing tool name and parameters.	Command Generation	To be Implemented
4	Validate and map JSON into safe CLI commands for supported tools (Nmap, Nikto, Dig).	Tool Adapter Mapping	To be Implemented

5	Execute validated commands within a sandboxed environment.	Sandbox Execution	To be Implemented
6	Analyze tool outputs and generate summarized results.	Result Analysis	To be Implemented
7	Display analyzed results through the frontend interface.	Result Visualization	To be Implemented
8	Log and store commands, outputs, and results in the database.	Data Management	To be Implemented
9	Detect and handle invalid or unsafe operations gracefully.	Error Handling	To be Implemented

Table 2 Functional Requirements

2.3 Non-Functional Requirements

Non-functional requirements describe how the system should perform rather than what it performs. These characteristics include security, performance, reliability, scalability, and usability.

Table 2.3: Non-Functional Requirements

S.No.	Non-Functional Requirement	Category
1	The system should respond within 3–5 seconds for command generation.	Performance
2	Multiple scan requests should be processed concurrently without noticeable delay.	Performance
3	All tool executions must occur in a sandboxed, isolated environment.	Security
4	The Tool Adapter must validate all CLI commands and block unauthorized flags or targets.	Security
5	All data and reports must be stored and transmitted securely using encryption.	Security
6	Modules must execute independently to prevent cascading failures.	Reliability
7	Automatic recovery and restart mechanisms must handle process failures.	Reliability
8	The system should easily support adding new tools (e.g., SQLMap, WPScan).	Scalability
9	The frontend should be responsive and intuitive, supporting all major browsers.	Usability
10	Results must be presented in a clear, human-readable format.	Usability
11	Modular backend architecture (FastAPI) must enable easy maintenance and debugging.	Maintainability

12	The system should integrate smoothly with Linux-based sandbox environments.	Compatibility

Table 3 Non Functional Requirements

2.4 Selected Functional Requirements

For the current prototype phase, the following functional requirements have been selected for implementation and demonstration:

Table 2.4: Selected Functional Requirements

S.No.	Functional Requirement	Type	Status
1	AI model interprets natural-language commands and generates structured JSON.	NLP & AI Processing	To be Implemented
2	Tool Adapter converts JSON into safe, validated CLI commands.	Command Mapping	To be Implemented
3	Commands execute in a secure, Docker-based sandbox environment.	Sandbox Execution	To be Implemented
4	Analyzer parses raw tool outputs into readable summaries.	Result Analysis	To be Implemented
5	Frontend displays the final report in an interactive dashboard.	Result Visualization	To be Implemented
6	Database stores user queries, tool results, and execution logs.	Data Management	To be Implemented
7	Customer login to the system	core	To be implemented

2.5 System Use Case Modeling

The use case model represents the interaction between the user, AI model, backend services, and sandbox. It provides a visual and descriptive overview of how data flows and actions are triggered.

Actors:

- User: Submits natural-language commands and views results.
- AI Model: Interprets the input and generates structured JSON.
- Backend (FastAPI): Validates requests and routes them to the Tool Adapter.
- Tool Adapter: Converts JSON data into safe, executable commands.
- Sandbox: Executes the tools securely and returns outputs.
- Analyzer: Processes raw tool results and summarizes findings.
- Database: Stores all logs, results, and user data.

Table 2.5: Use Case Summary

Use Case	Description	Actors Involved
Submit Scan Request	User enters a natural-language command through the UI.	User, Frontend
Interpret Command	AI model interprets user input and produces JSON output.	AI Model
Generate Safe Command	Backend validates JSON and maps it to a safe CLI command.	Backend, Tool Adapter
Execute Scan	Runs validated commands in the sandboxed environment.	Sandbox
Analyze Results	Processes and summarizes scan output for readability.	Analyzer
View Results	Displays final summarized results on the dashboard.	User, Frontend
Save Logs	Stores command history, outputs, and metadata in the database.	Backend, Database

Table 4 Use Case Summary

2.6 System Sequence Diagram

The system sequence diagram (SSD) illustrates how each component communicates step-by-step during a penetration testing request. It highlights the operational flow from user input to output display.

Relationships:

- The User interacts with the Frontend, which communicates with the Backend API.
- The Backend utilizes the AI Model to interpret commands and the Tool Adapter to create executable instructions.
- The Sandbox securely executes the tools, returning data to the Analyzer.
- The Analyzer stores results in the Database and sends structured reports to the Frontend for user viewing.

1) AI model interprets natural-language commands and generates structured JSON

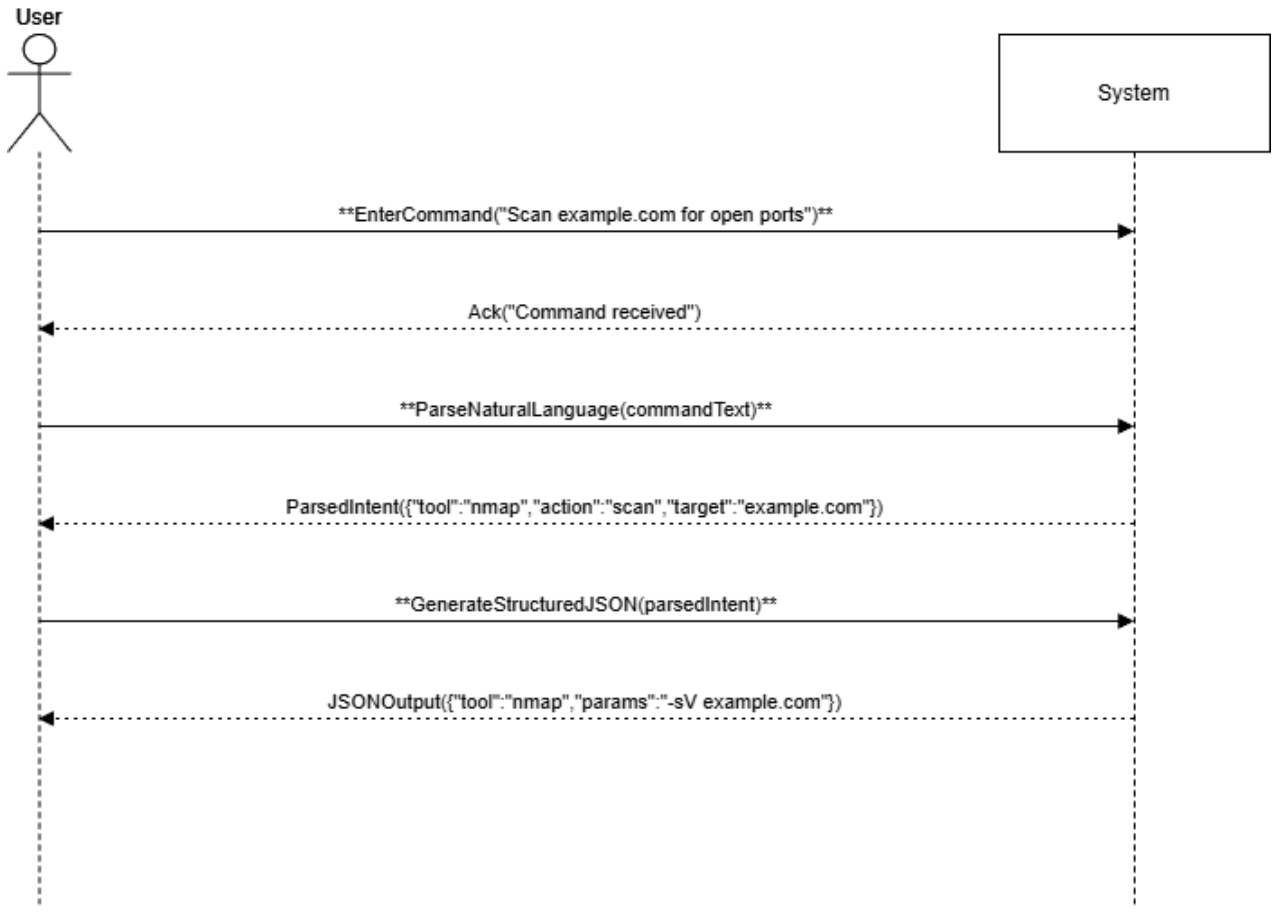


Figure 1 JSON generator

2) Tool Adapter converts JSON into safe, validated CLI commands

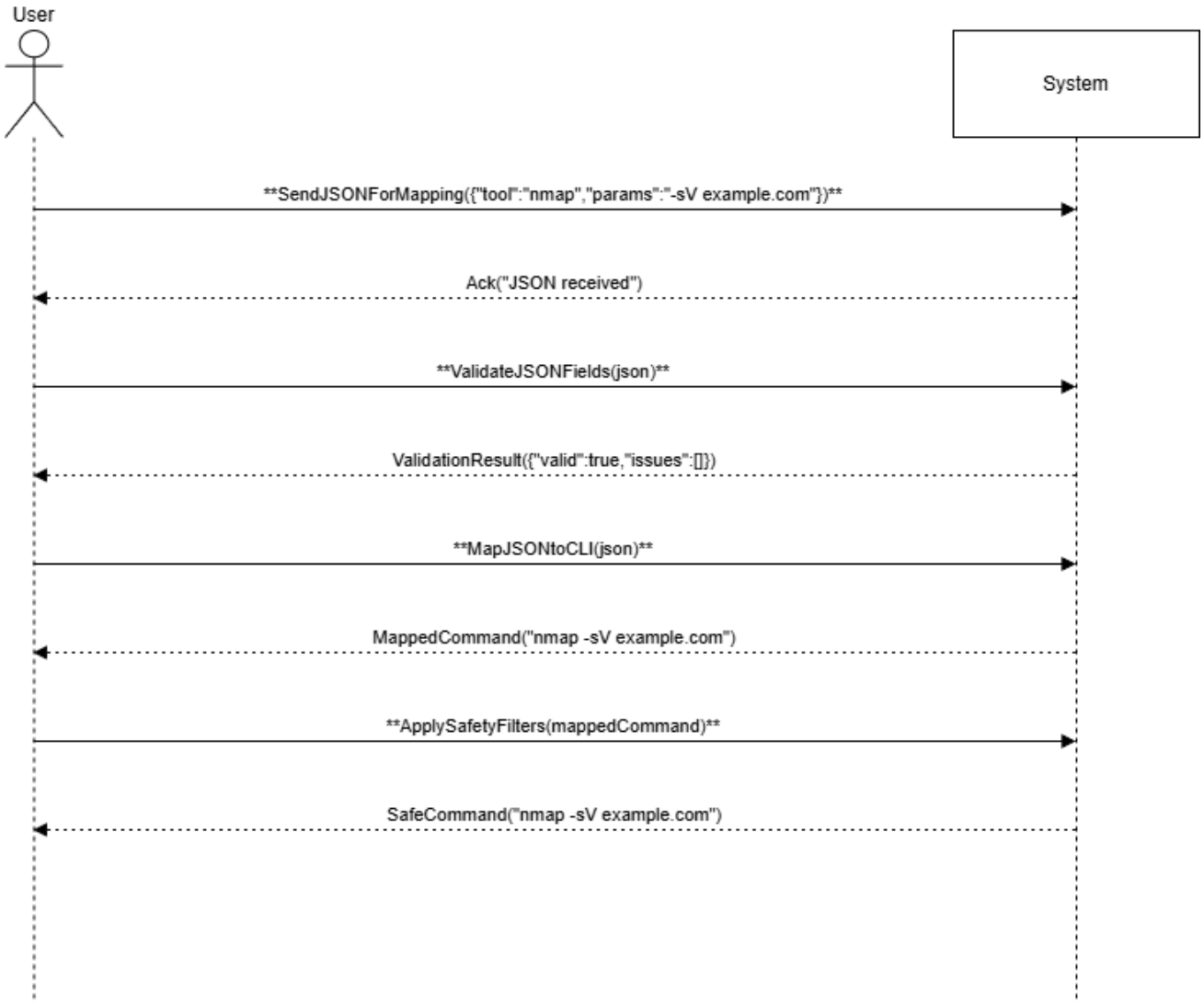


Figure 2 command generator

3) Commands execute in a secure, Docker-based sandbox environment

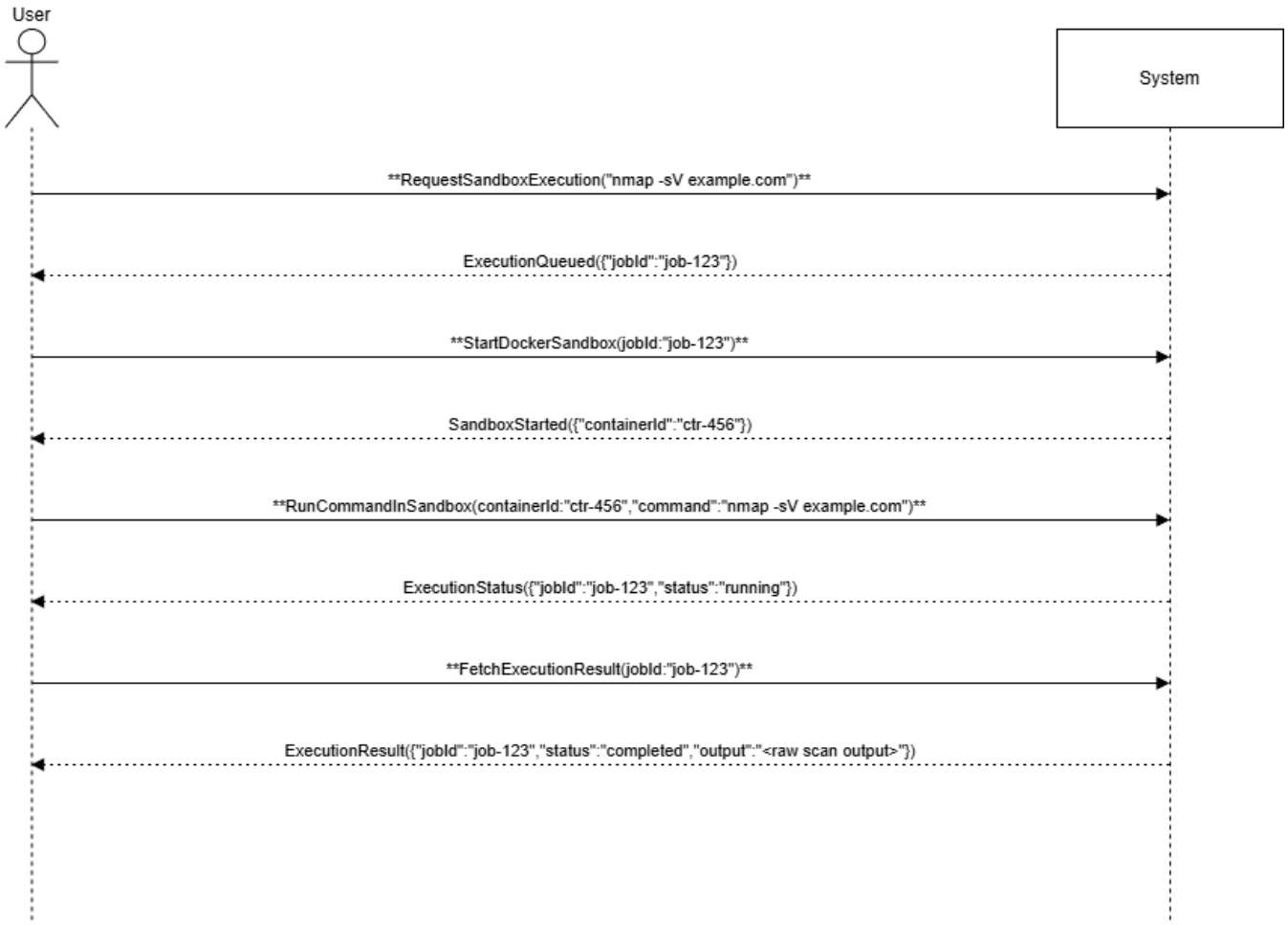


Figure 3 command execution in sandbox

4) Analyzer parses raw tool outputs into readable summaries

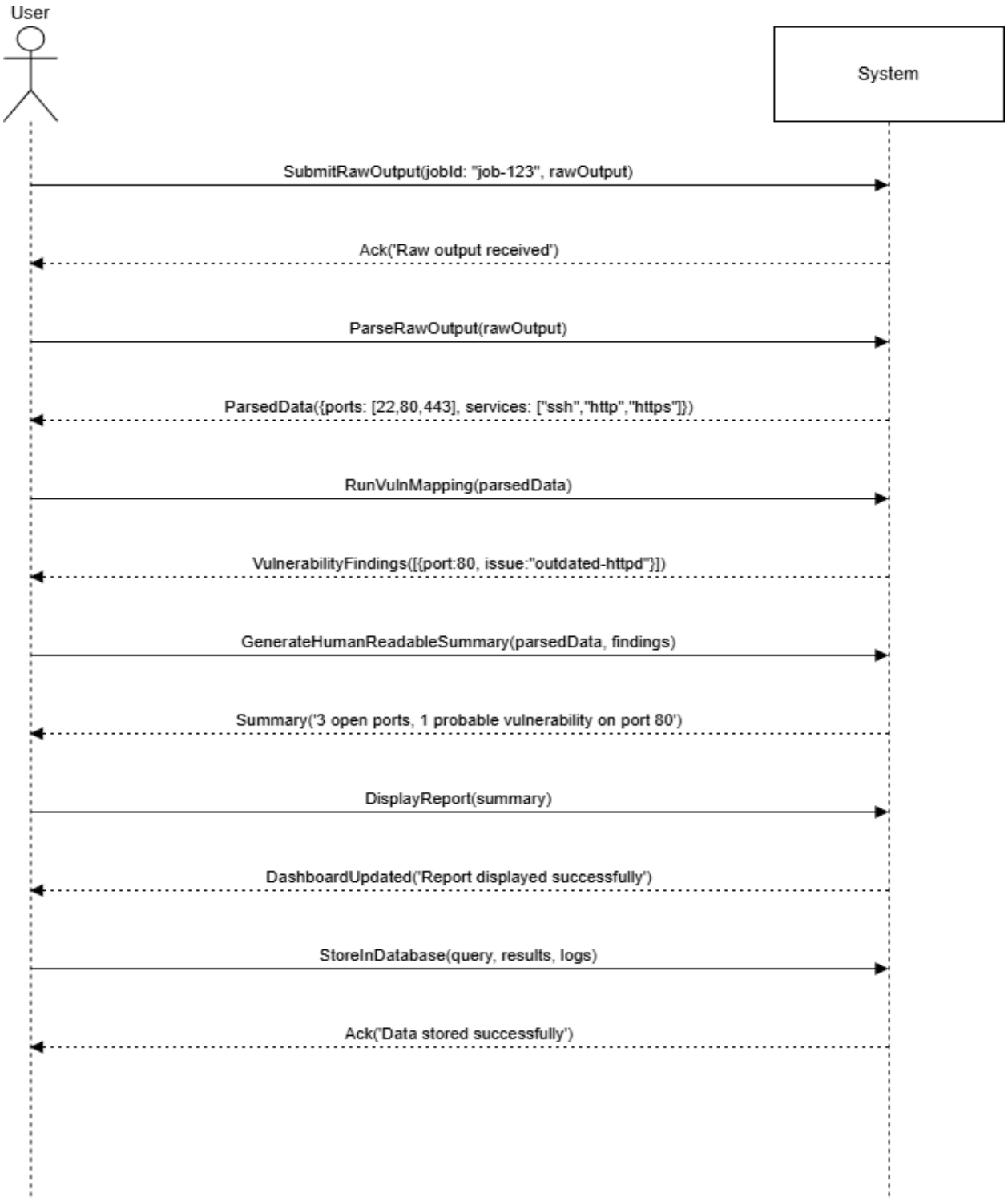


Figure 4 Analyzer

5) Frontend displays the final report in an interactive dashboard

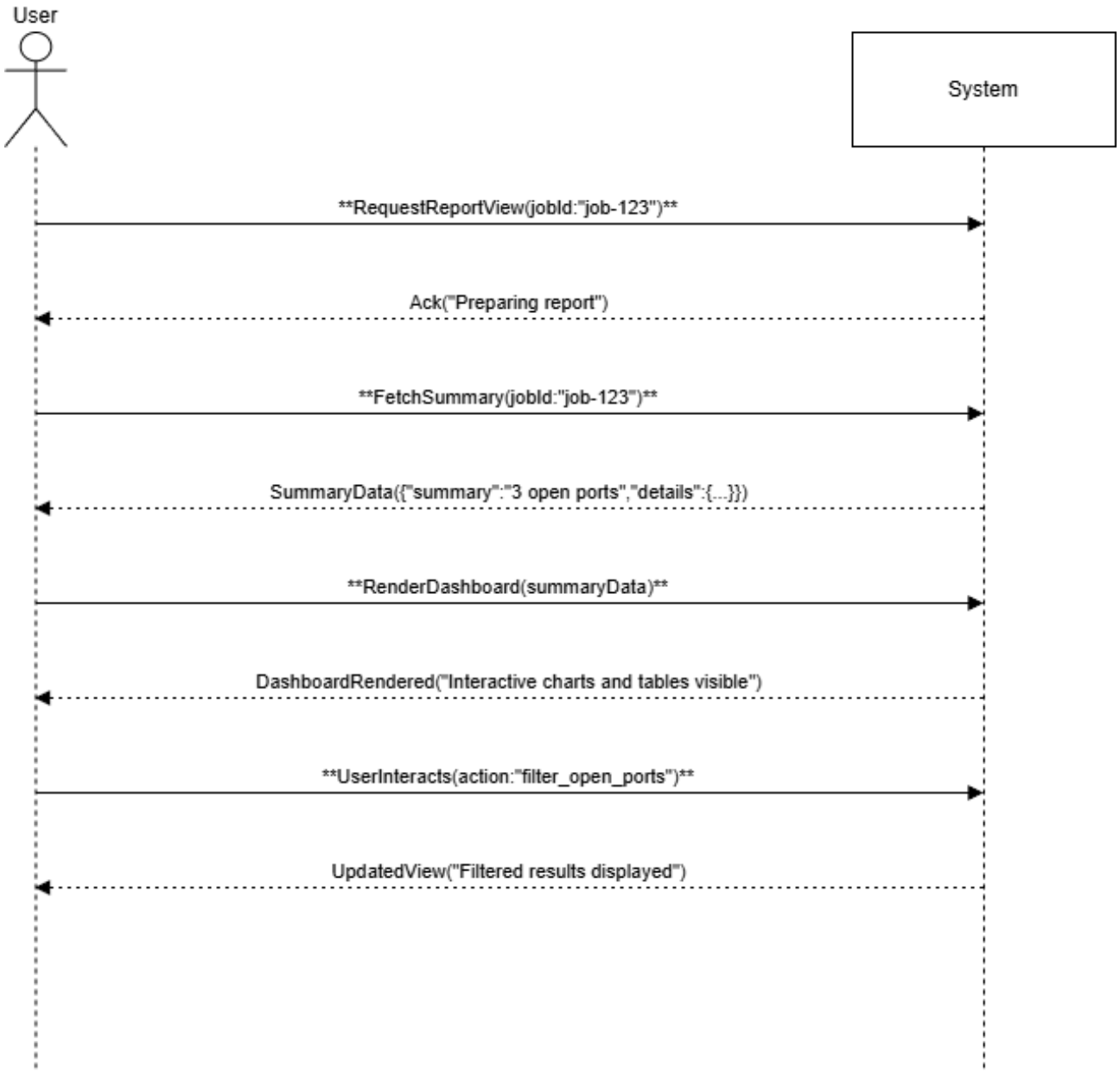


Figure 5 Display Final Report

6) Database stores user queries, tool results, and execution logs

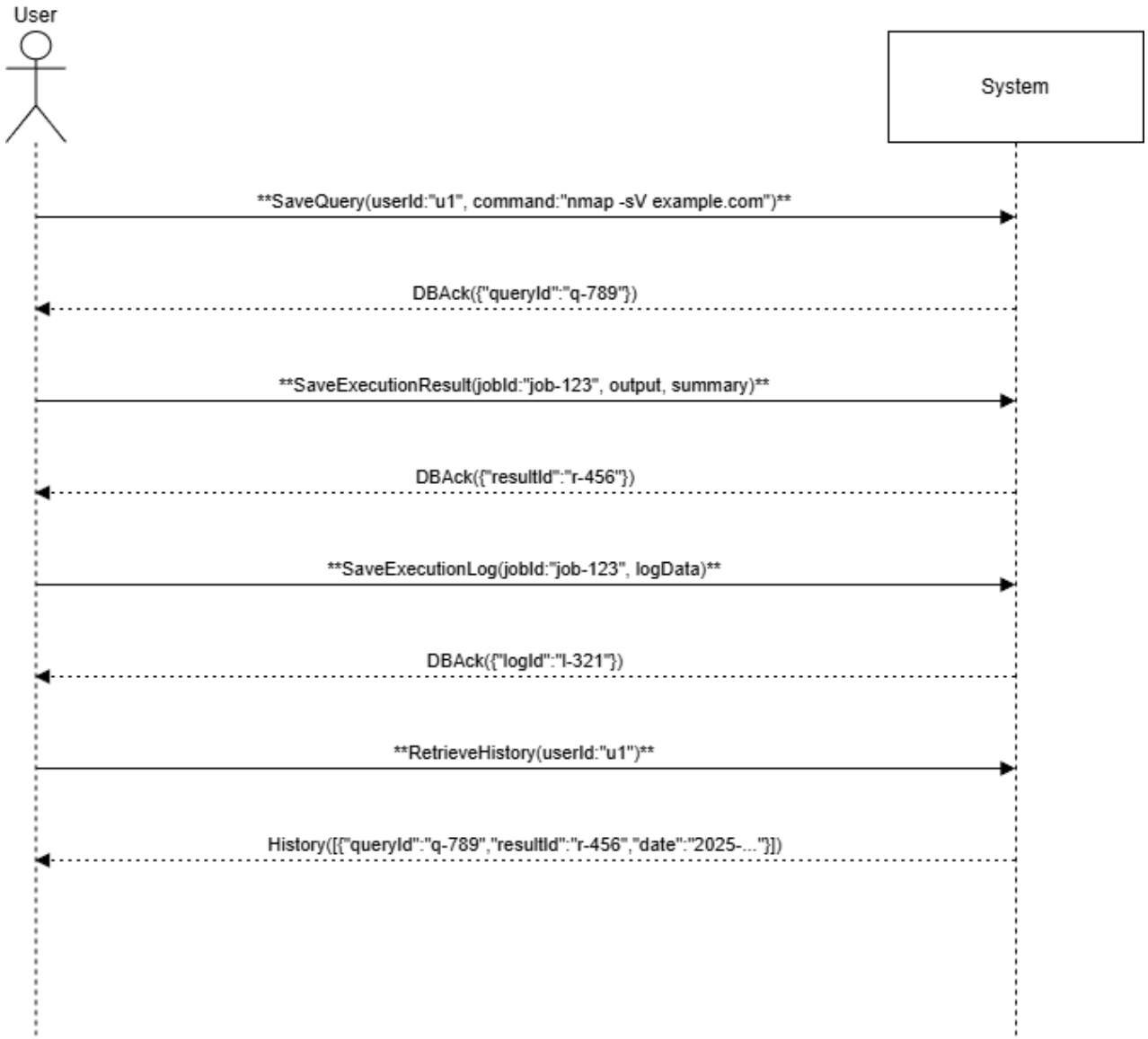


Figure 6 Database

2.7 Domain Model

The domain model defines the key entities and their relationships within the AI-Driven Penetration Testing Assistant. It provides a conceptual understanding of how components interact to deliver secure, automated testing.

Table 2.6: Domain Model Entities

Entity	Description
User	Initiates scan requests and views summarized reports.
AI Model	Interprets natural-language input and generates structured JSON output.
Backend API	Routes data between frontend, AI model, and sandbox modules.
Tool Adapter	Validates JSON and converts it into secure CLI commands.
Sandbox	Executes tools (Nmap, Nikto, Dig) in an isolated environment.
Analyzer	Processes raw outputs and produces readable summaries.
Database	Stores commands, scan results, and logs.
Tools (Nmap, Nikto, Dig)	Perform scanning, enumeration, and vulnerability detection.

Table 5 Domain Model Entities

Domain model:

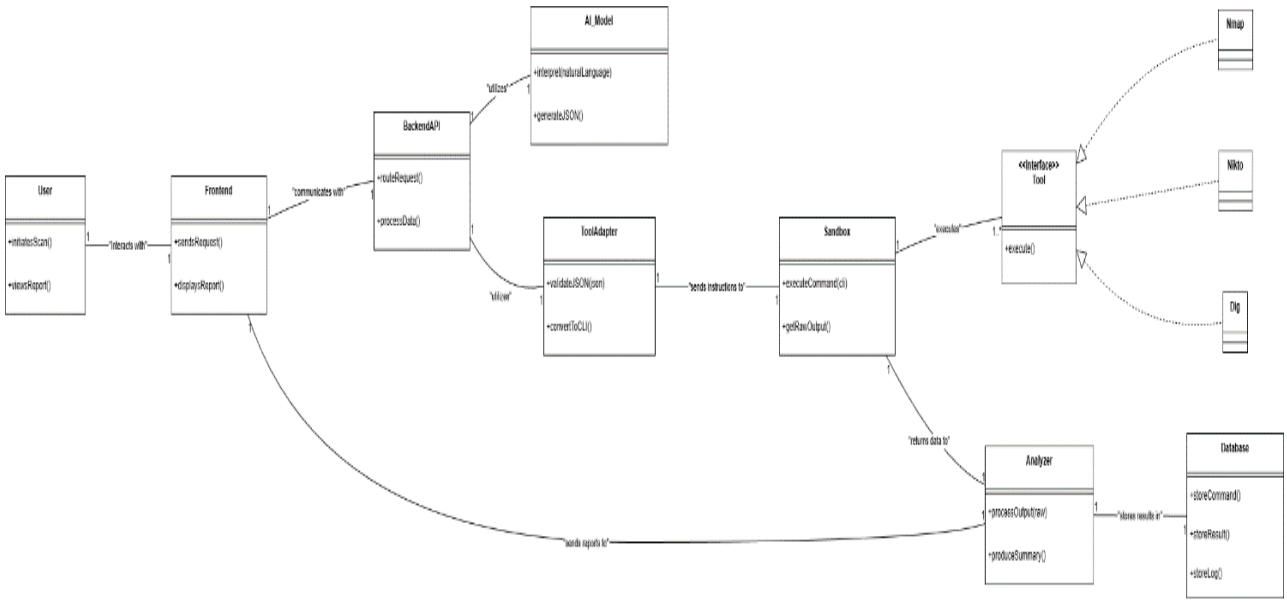


Figure 7 Domain Model

Use case:

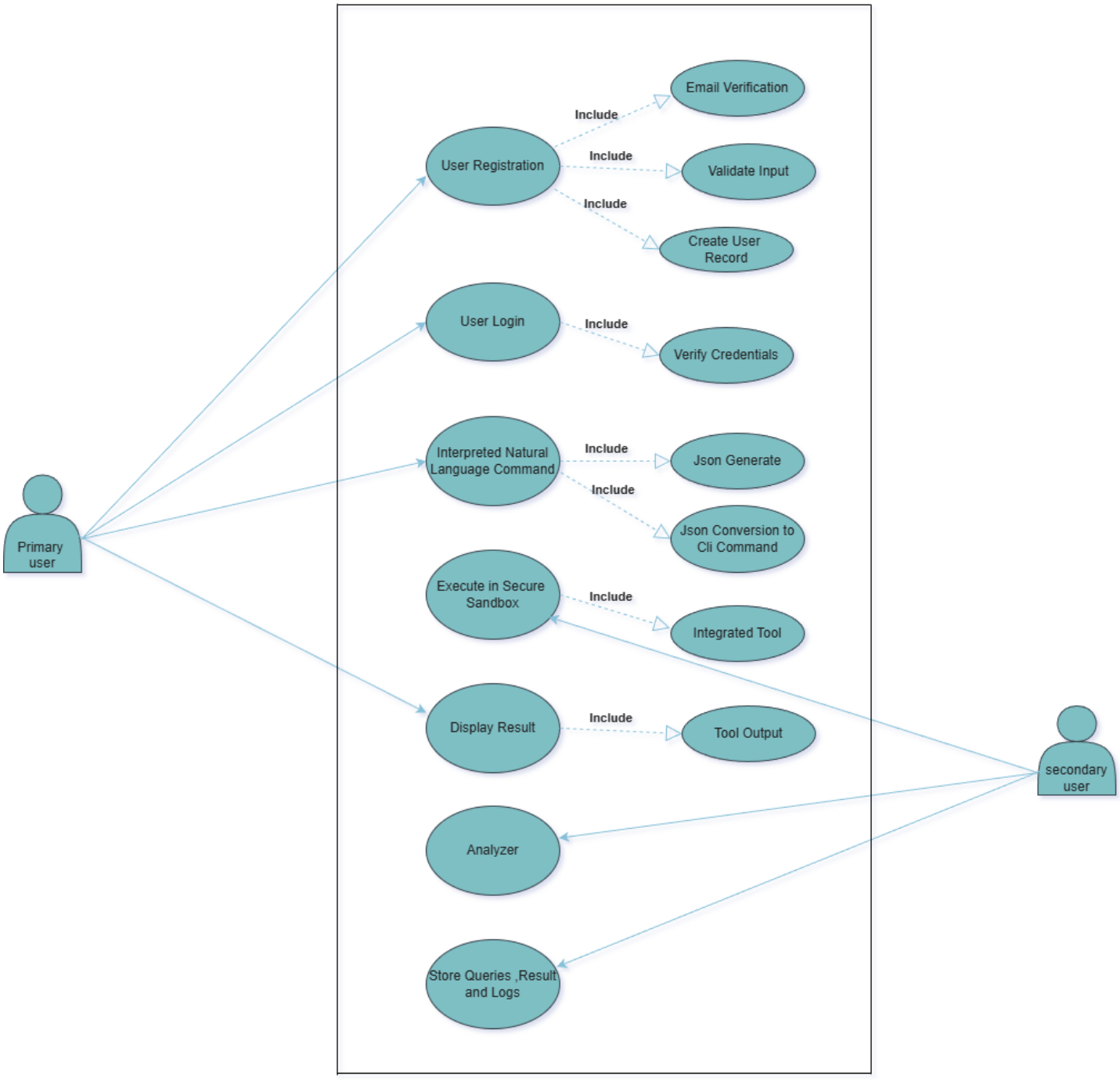


Figure 8 Use case Diagram

Chapter 3

3.1 System Design

This chapter provides a comprehensive and professionally structured description of the system architecture, components, operational workflow, and safety mechanisms of the AI-Driven Penetration Testing Assistant. The chapter defines how the system converts a natural-language cybersecurity query into a safe, validated, and reproducible penetration-testing task. The design emphasizes modularity, security, scalability.

3.6 Layered Architecture

The system adopts a three-tier architecture to ensure a clear separation of concerns and to maintain scalability, flexibility, and maintainability. Each layer encapsulates its functionality and communicates through secure, well-defined interfaces.

3.2.1 Presentation Layer

The Presentation Layer is developed using React.js and provides the User Interface (UI) for all user interactions. It is designed with usability principles to support students, cybersecurity learners, and professional analysts. It delivers a clean, responsive interface and performs the following functions:

- Accepts natural-language penetration testing queries
- Displays real-time scan status and progress indicators
- Presents structured scan results, summaries, and technical details
- Manages user authentication (login, signup, role assignment)
- Shows scan history, timestamps, and associated metadata
- Provides error feedback and meaningful validation messages

The UI communicates with the backend via secure HTTPS-based REST APIs using a token-based authentication mechanism.

3.2.2 Business Logic Layer

The Business Logic Layer represents the intelligence core of the system, implemented using Python (FastAPI/Flask). It is responsible for processing user queries, generating safe commands, executing operations in isolation, and interpreting the resulting output. This layer contains the following key modules:

i. LLM Interpreter:

Converts user input into a structured and machine-readable JSON specification using Large Language Model inference. The JSON contains tool selection, scanning parameters, and target information.

ii. JSON Validator:

Rigorously validates the LLM output to ensure it conforms to security and syntax constraints. This includes IP validation, CIDR restrictions, safe port ranges, and whitelisted flags.

iii. Tool Adapter (Command Builder)

Generates secure, template-based command-line instructions from validated JSON. It ensures that only approved tools (e.g., Nmap, Nikto, Dig) and safe flags are used.

iv. Execution Sandbox

Runs all generated commands in a controlled, isolated execution environment. This sandbox restricts network access, CPU/memory resources, and filesystem exposure, preventing system misuse.

v. Output Analyzer

Processes raw tool output, extracts actionable findings, and produces structured results, including vulnerability indicators, summaries, and remediation suggestions.

3.2.3 Database Layer

The Database Layer, implemented using PostgreSQL or Firebase, provides secure and persistent storage for all system data. It maintains user records, scan logs, JSON payloads, command histories, and audit trails. Proper indexing ensures efficient retrieval of historical scan results, supporting analytical and academic reporting.

3.7 Software Architecture

The system architecture follows a modular, API-driven design. Each component is loosely coupled, enabling easy maintenance and future extensions. The data flow proceeds as follows:

This architecture strengthens scalability, supports concurrent scanning operations, and ensures strict safety through multi-level validation and isolation.

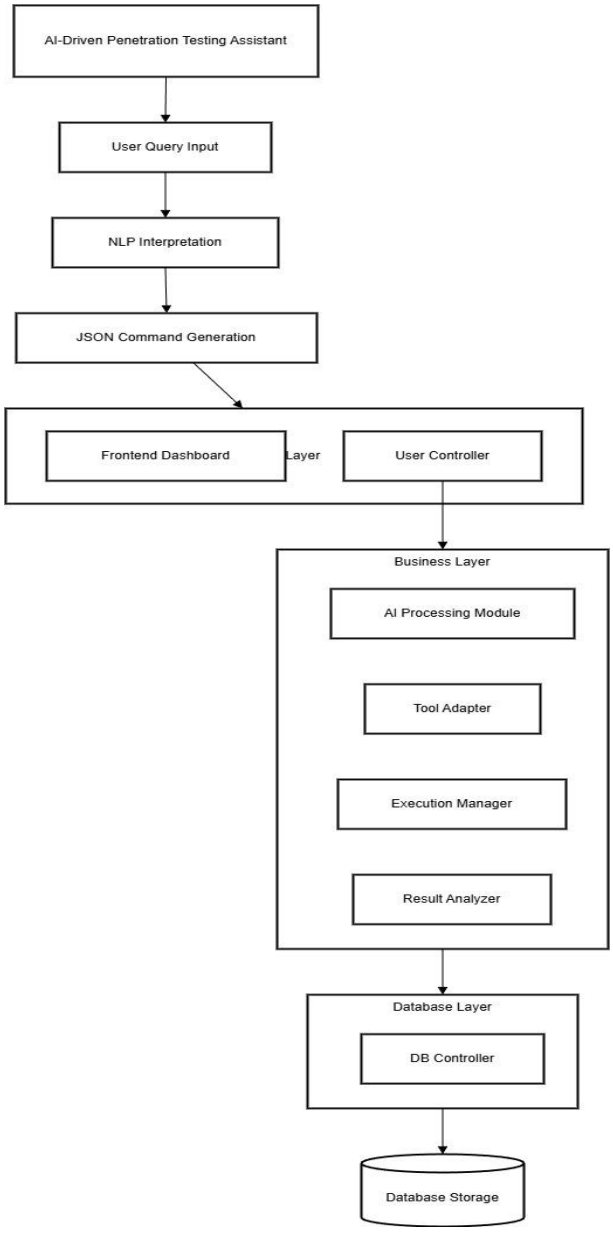


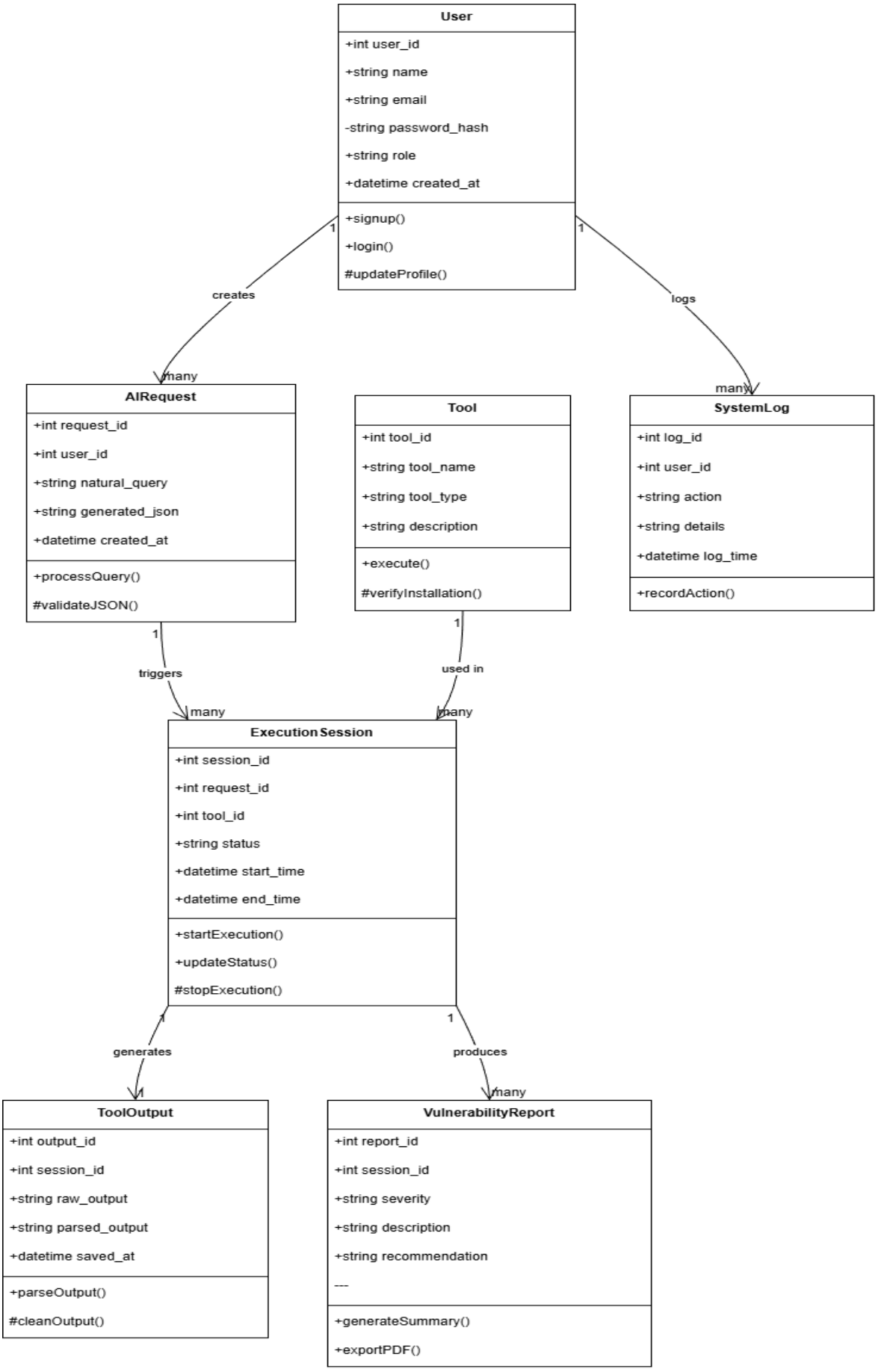
Figure 9 System Architecture Diagram

3.4 Class Diagram Overview

- The system's backend logic is represented through the following primary classes:

• **User:** Manages authentication, roles, and access control.
• **Scan-Request:** Stores the initial natural-language input and its corresponding JSON structure.
• **AI-Model:** Handles LLM communication and prompt engineering.
• **JSON-Validator:** Validates each field of the JSON response.
• **Tool-Adapter:** Converts validated JSON into safe CLI commands.
• **Sandbox-Executor:** Executes commands within an isolated container.
• **Analyzer**: Interprets raw output and generates structured findings.
• **Scan-Result:** Stores final processed results, summaries, and vulnerability data.

These classes interact sequentially to transform user intent into secure, actionable penetration-testing results.


User
+int user_id
+string name
+string email
-string password_hash
+string role
+datetime created_at
+signup()
+login()
#updateProfile()
1
creates
many
1
logs
many
AIRequest
+int request_id
+int user_id
+string natural_query
+string generated_json
+datetime created_at
+processQuery()
#validateJSON()
Tool
+int tool_id
+string tool_name
+string tool_type
+string description
+execute()
#verifyInstallation()
SystemLog
+int log_id
+int user_id
+string action
+string details
+datetime log_time
+recordAction()
1
triggers
many
1
used in
many
ExecutionSession
+int session_id
+int request_id
+int tool_id
+string status
+datetime start_time
+datetime end_time
+startExecution()
+updateStatus()
#stopExecution()
1
generates
1
1
produces
many
ToolOutput
+int output_id
+int session_id
+string raw_output
+string parsed_output
+datetime saved_at
+parseOutput()
#cleanOutput()
VulnerabilityReport
+int report_id
+int session_id
+string severity
+string description
+string recommendation

+generateSummary()
+exportPDF()

3.5 Sequence Diagram Explanation

The sequence of operations executed during a typical scan ensures controlled, traceable, and validated processing. Key steps include:

1. User submits a natural-language request.
2. UI forwards it to the backend API.
3. Backend sends the request to the LLM.
4. LLM returns structured JSON describing the scan.
5. JSON Validator verifies safety and compliance.
6. Tool Adapter generates a secure command.
7. Execution Sandbox runs the command.
8. Raw output is processed by the Analyzer.
9. Results are stored in the database.
10. UI retrieves and displays the structured results.

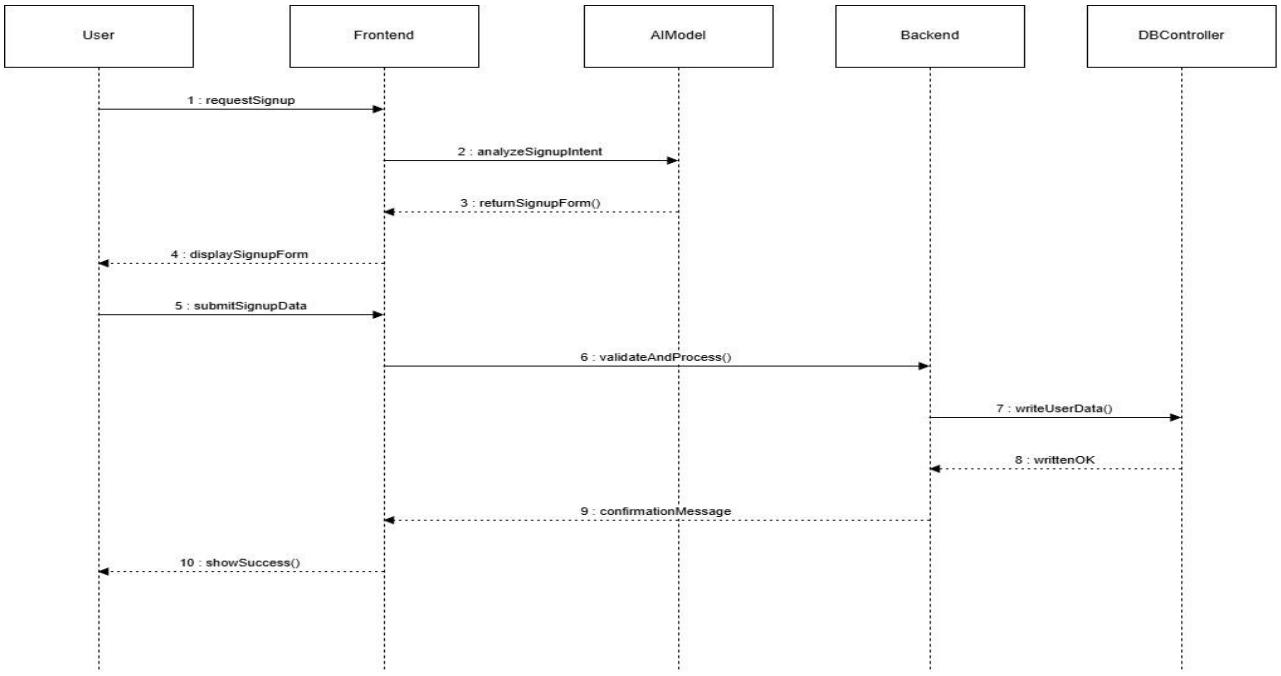


Figure 10 Sign Up

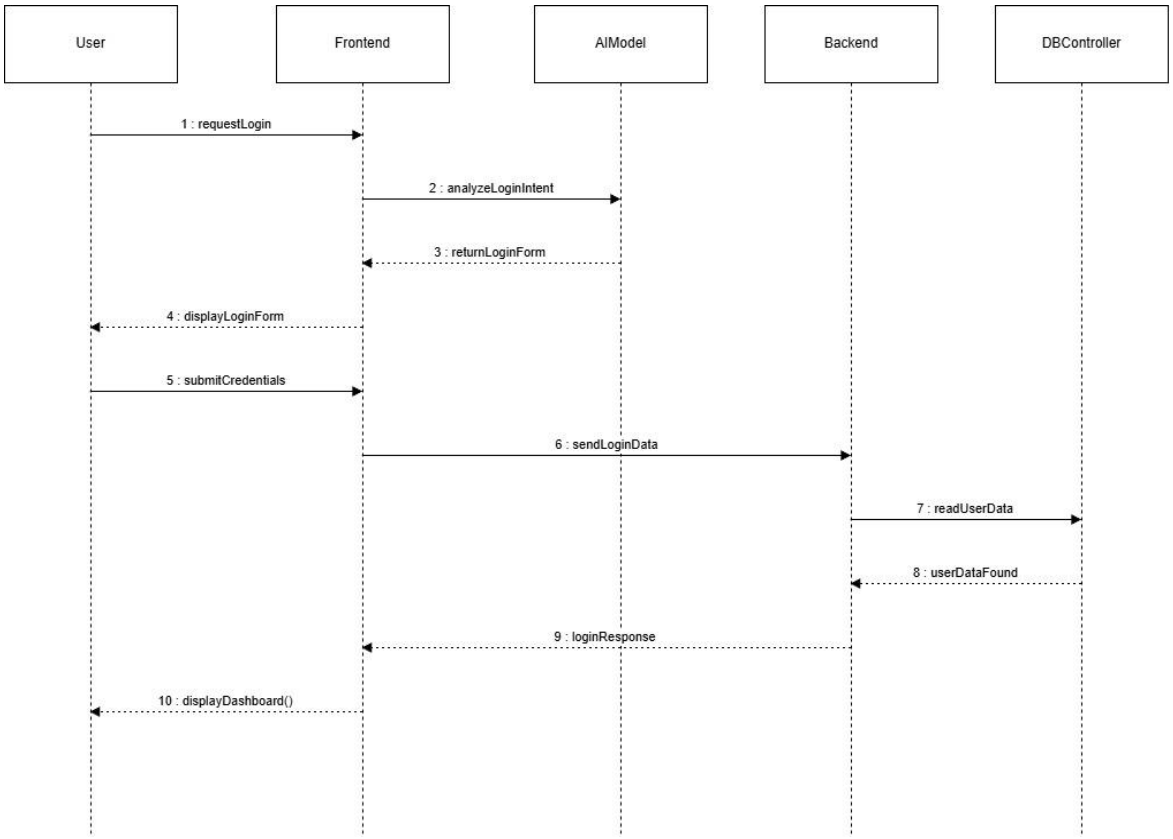


Figure 11 Login

3.6 Entity Relationship Diagram (ERD)

The ERD defines relationships among system entities, ensuring efficient storage and traceability.

Key entities include:

- User
- ScanRequest
- ScanResult
- ToolLog
- AuditTrail

A user may submit multiple scan requests; each request corresponds to a single scan result and may produce multiple tool logs. Audit trails ensure oversight and compliance.

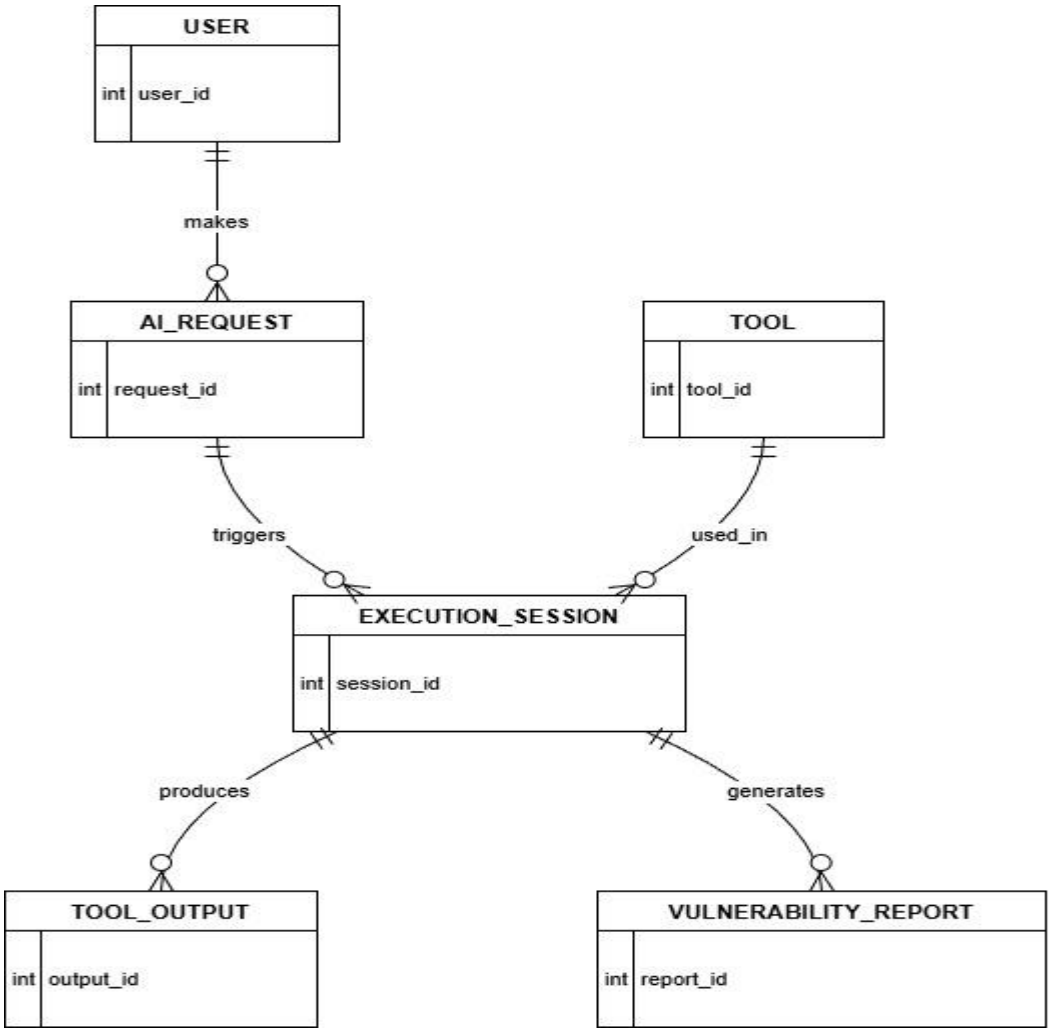


Figure 12 ERD Diagram

3.7 Database Schema

The database schema is divided into well-structured tables designed for clarity, normalization, and consistency:

• Users: Credentials, roles, timestamps

• Scan-Requests: Original query, JSON payload, tool selected

• Scan-Results: Final summaries, findings, severity ratings

• Tool-Logs: Executed commands, raw outputs, execution timestamps

• Audit-Trail: Logged user actions for security tracking

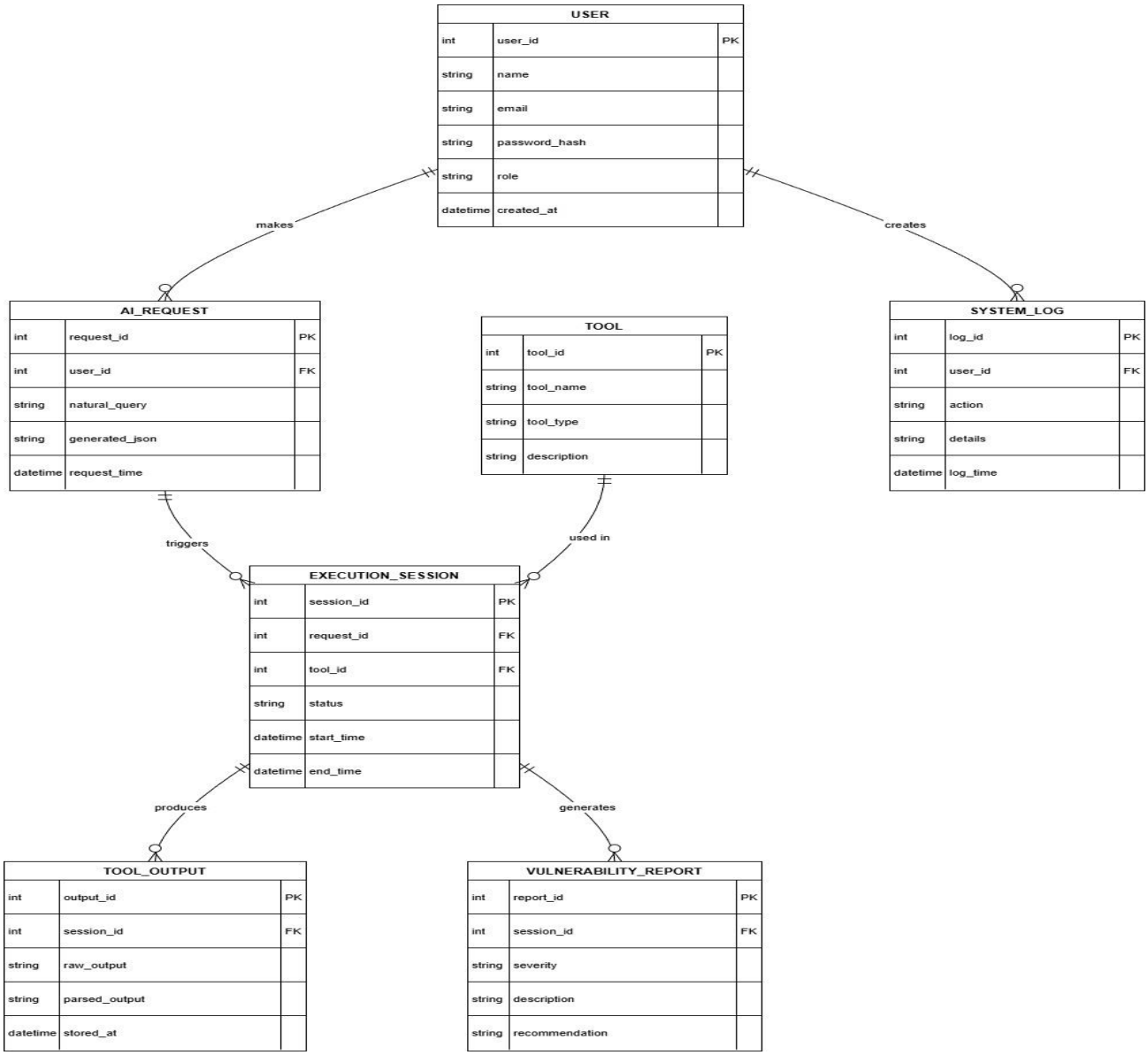


Figure 13 Database Schema

3.8 User Interface Design

The UI focuses on clarity, accessibility, and academic usability. Key screens include:

- Login / Signup
- Dashboard Overview
- New Scan Interface
- Execution Status View
- Result Summary Interface
- Scan History Page

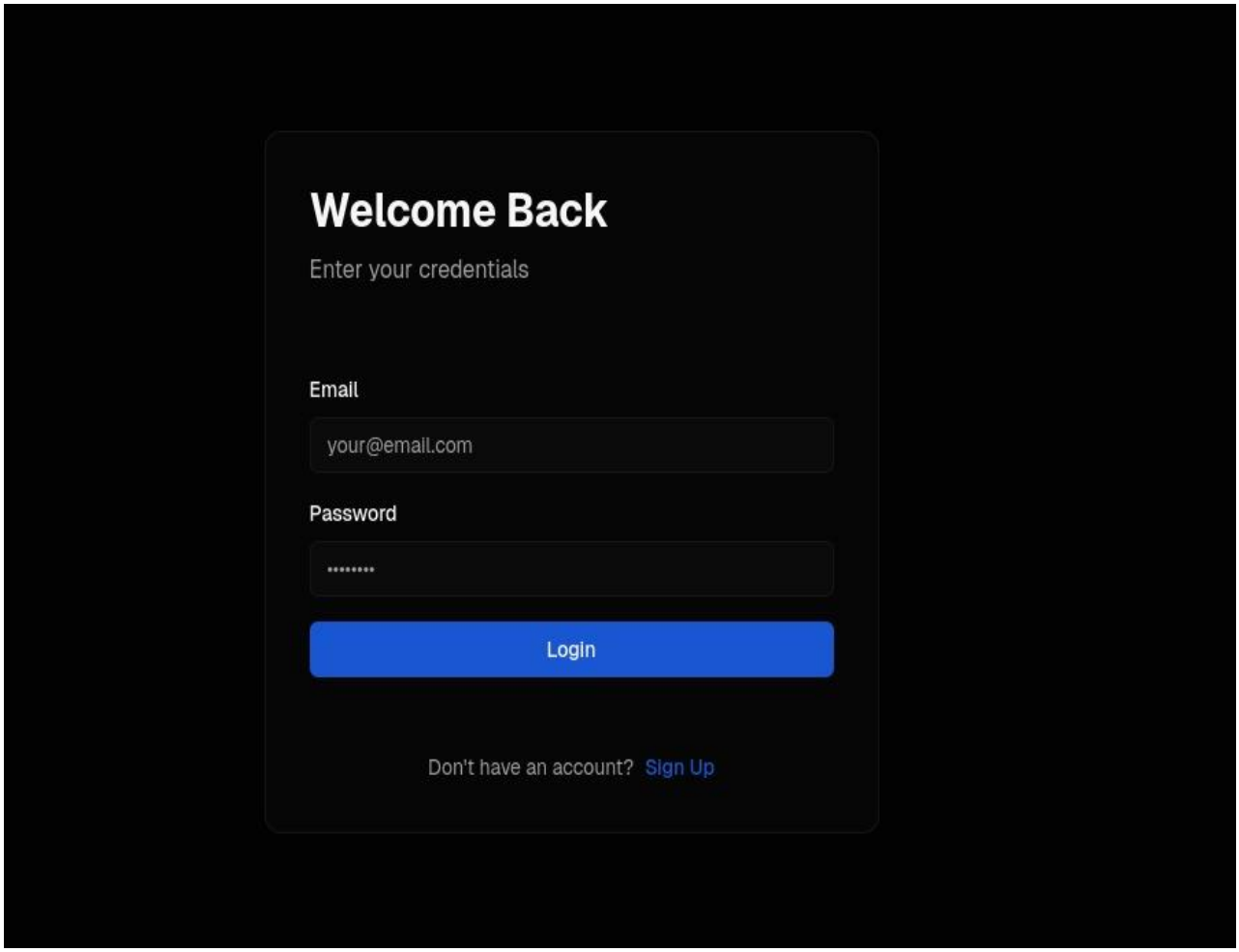


Figure 14 Login Page

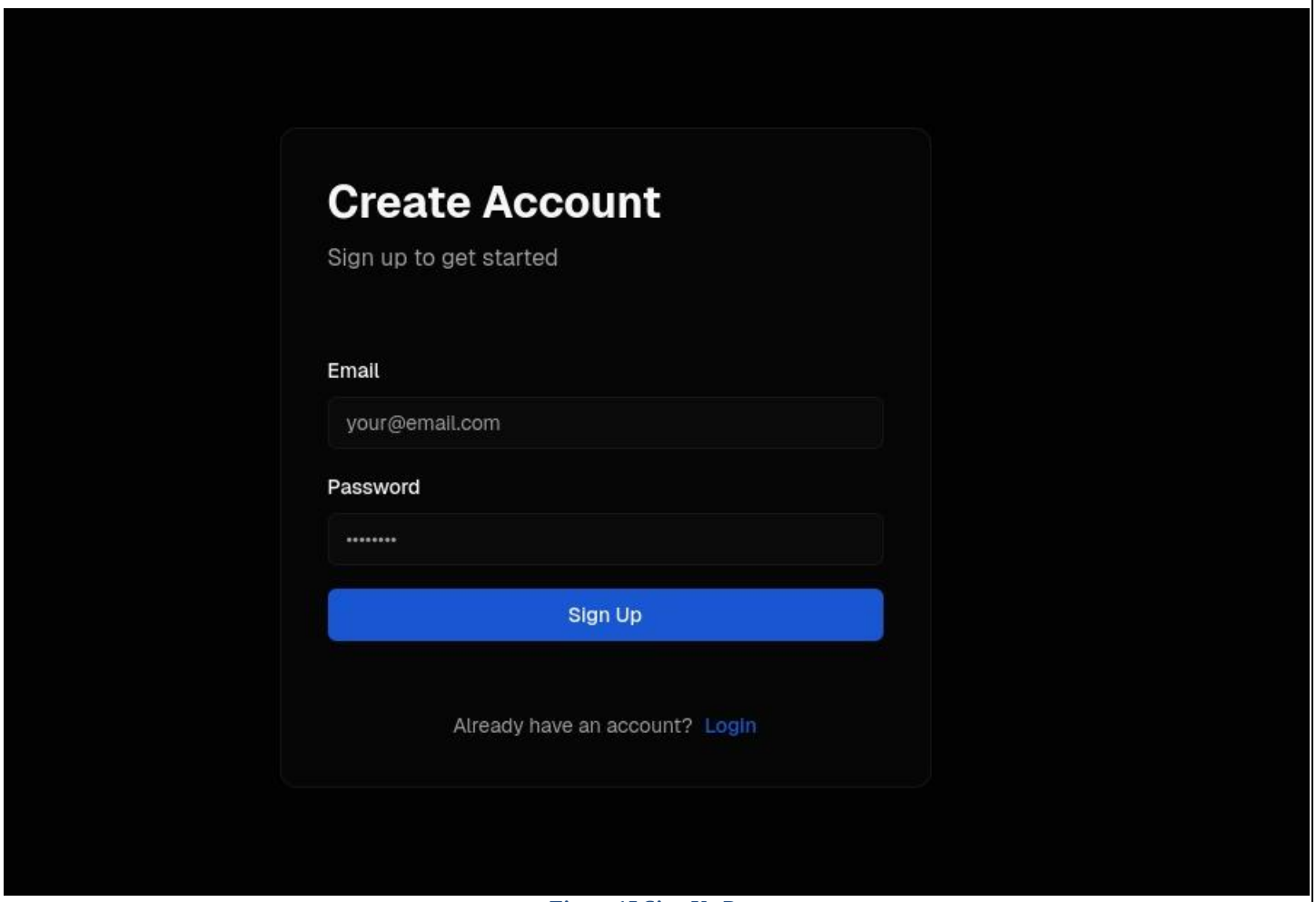


Figure 15 Sign Up Page

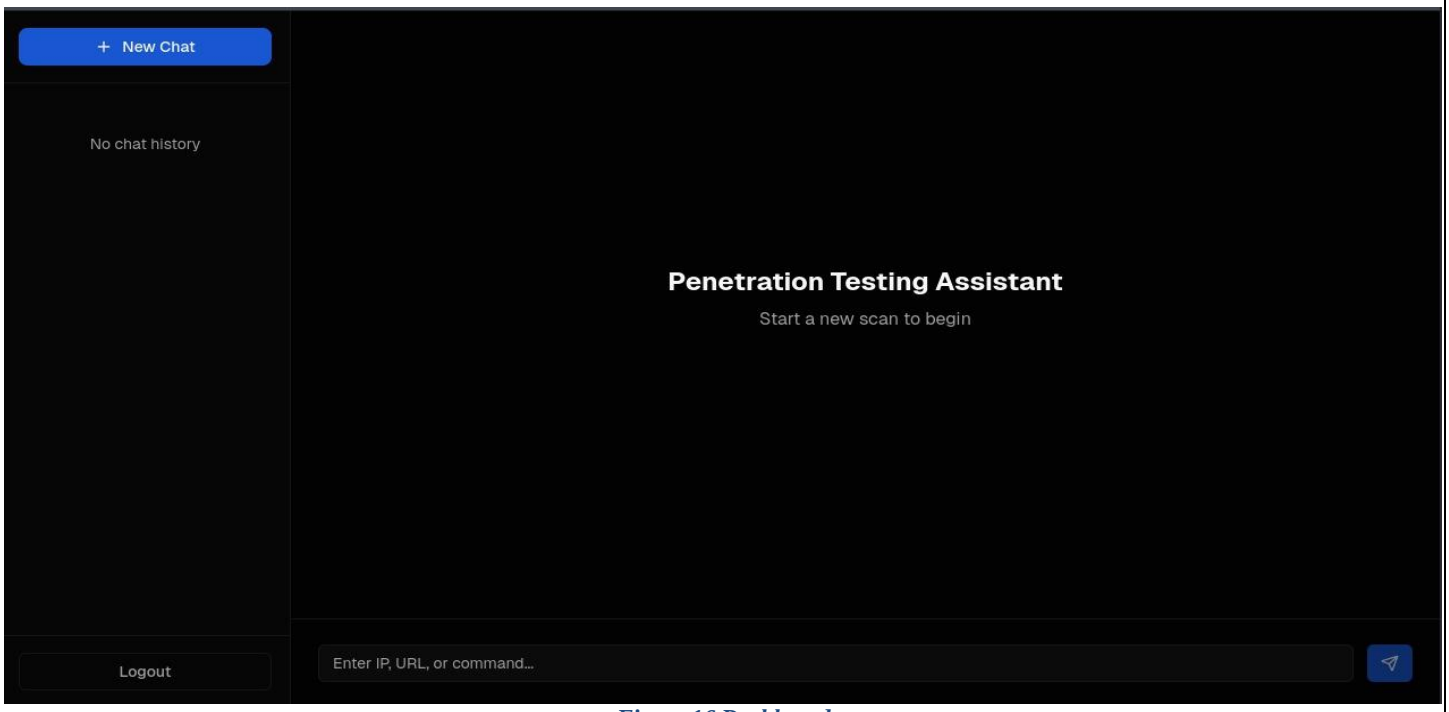


Figure 16 Dashboard